Ensemble Learning Techniques to Detect Fake News

12/5/2025

Abstract

This project aims to detect fake news using ensemble learning techniques by analyzing text data from news articles. The workflow includes data loading, preprocessing, exploratory analysis, and preparation for machine learning models.

Team members

1-Ammar Mohammed Aljohari

2-Mohammed Essam Elsayed

3-Mohammed Rezk Abdo

4-Abdulrahman Sabry

Content

Introduction

This project aims to detect fake news using ensemble learning techniques by analyzing text data from news articles. The workflow includes data loading, preprocessing, exploratory analysis, and preparation for machine learning models.

Dataset

- **Source**: fake.csv containing 12,999 entries with features like text, type, author, title, and metadata (e.g., spam_score, domain_rank).
- **Target Variable**: type (categories include bias, conspiracy, hate, etc.).
- **Key Columns**:
    - text: Article content (primary feature for analysis).
    - type: Label indicating the news category

Data Preprocessing

- Retain only relevant columns: text and type.
- Clean the text column:
    - Remove non-word characters using regex.
    - Convert text to lowercase for uniformity.

Exploratory Data Analysis (EDA)

- Visualize the distribution of news types:
    - Majority of articles are labeled bias, indicating potential class imbalance.
    - Other categories include conspiracy, hate, and satire.

Next Steps (Hypothesized)

Text Vectorization

- Convert text data into numerical features using techniques like TF-IDF or word embeddings.

Ensemble Learning

- Implement models such as:
    - **Random Forest**: Combines multiple decision trees.
    - **Gradient Boosting**: Builds sequential models to correct errors.
    - **Voting Classifier**: Aggregates predictions from diverse models.
- Address class imbalance using:
    - **SMOTE** (Synthetic Minority Oversampling Technique).
    - Class weighting in models.

Evaluation Metrics

- Use **accuracy, precision, recall, F1-score** to assess performance.
- Generate a confusion matrix to evaluate per-class performance.

Challenges

- **Class Imbalance**: Dominance of bias articles may skew model performance.
- **Text Noise**: Raw text requires robust cleaning (e.g., handling URLs, stopwords).
- **Model Interpretability**: Ensemble methods can be complex; techniques like SHAP values might help explain predictions.

This script evaluates and compares the performance of **three classification models**:

- **Stacking**
- **Gradient Boosting**
- **Random Forest**

It uses **four key metrics** from classification tasks:

- **Accuracy** – How often the model gets predictions right overall.

- **Precision (macro avg)** – How many selected items are relevant (averaged across classes).
- **Recall (macro avg)** – How many relevant items are selected (averaged across classes).
- **F1-Score (macro avg)** – The harmonic mean of precision and recall.

Workflow Summary:

1. It initializes an empty table results_df to hold metrics.
2. For each model:
    - It gets predictions.
    - Generates a classification report.
    - Extracts accuracy, precision, recall, and F1-score.
    - Adds the metrics to the results table under the model's name.
3. The final result is a clean **DataFrame** where each row is a model and each column is a metric.

Why This Is Useful:

This method provides a structured way to:

- Evaluate multiple models consistently.
- Compare performance side-by-side.
- Identify which model performs best across various metrics.

	Accuracy	Precision	Recall	F1-Score
Stacking	0.914705	0.691619	0.434827	0.517097
Gradient Boosting	0.895407	0.523937	0.360752	0.398239
Random Forest	0.902354	0.703973	0.220997	0.276273